



LeetCode 1838: Frequency of the Most Frequent Element

1. Problem Title & Link

Title: LeetCode 1838: Frequency of the Most Frequent Element

Link: <https://leetcode.com/problems/frequency-of-the-most-frequent-element/>

2. Problem Statement (Short Summary)

You are given:

- An integer array `nums`
- An integer `k`

In one operation, you can:

Increase any element by 1

You can perform at most:

`k` operations

Return the maximum possible frequency of any element after performing at most `k` operations.

3. Examples (Input → Output)

Example 1

Input:

`nums = [1,2,4]`

`k = 5`

Output:

3

Explanation:

Use 5 operations:

`1 → 4` (3 operations)

`2 → 4` (2 operations)

Result:

`[4,4,4]`

Frequency = 3

Example 2

Input:

`nums = [1,4,8,13]`

`k = 5`

Output:

2



Example 3

Input:

nums = [3,9,6]

k = 2

Output:

1

4. Constraints

- $1 \leq \text{nums.length} \leq 10^5$
- $1 \leq \text{nums}[i] \leq 10^5$
- $1 \leq k \leq 10^5$

Need better than:

$O(n^2)$

5. Core Concept (Pattern / Topic)

Sorting + Sliding Window

Related Topics:

- Two Pointers
- Window Expansion
- Greedy

6. Thought Process (Step-by-Step Explanation)

Brute Force

For every element:

Assume it becomes the target.

Calculate cost to convert all previous elements into it.

Complexity:

$O(n^2)$

Too slow.

Key Observation

After sorting:

nums = [1,2,4]

To maximize frequency, we should make smaller numbers equal to a larger number.

Example:

$1 \rightarrow 4$

$2 \rightarrow 4$



Possible.

But:

$4 \rightarrow 1$

Not possible because only increments are allowed.

Sliding Window Idea

Suppose current window:

[1,2,4]

Target:

4

Cost required:

$(4-1)+(4-2)+(4-4)$

$= 3+2+0$

$= 5$

If:

$\text{cost} \leq k$

window is valid.

Otherwise:

Shrink from left

Efficient Cost Formula

Window:

left ... right

Target:

$\text{nums}[\text{right}]$

Cost:

$\text{nums}[\text{right}] * \text{windowSize} - \text{windowSum}$

This computes total increments needed.

7. Visual / Intuition Diagram (ASCII Diagram)

Sorted Array:

1 2 4 8

L R

Target:

4

Need:

$1 \rightarrow 4 = 3$

$2 \rightarrow 4 = 2$

$4 \rightarrow 4 = 0$

Total:

5

Valid if:

$5 \leq k$

8. Pseudocode (Language Independent)

```
sort nums

left = 0
windowSum = 0
answer = 1

for right:

    add nums[right]

    while cost > k:

        remove nums[left]
        left++

    answer = max(answer, window size)

return answer
Cost:
nums[right] * windowSize - windowSum
```

9. Code Implementation

 Python

```
class Solution:

    def maxFrequency(self, nums, k):

        nums.sort()

        left = 0
        windowSum = 0
        answer = 1

        for right in range(len(nums)):

            windowSum += nums[right]
```

```
        while (
            nums[right] *
            (right - left + 1)
            - windowSum
        ) > k:

            windowSum -= nums[left]
            left += 1

        answer = max(
            answer,
            right - left + 1
        )

    return answer
```

✓ Java

```
import java.util.*;

class Solution {

    public int maxFrequency(
        int[] nums,
        int k
    ) {

        Arrays.sort(nums);

        int left = 0;
        int answer = 1;

        long windowSum = 0;

        for (int right = 0;
            right < nums.length;
            right++) {

            windowSum += nums[right];

            while (
                (long) nums[right]
                * (right - left + 1)
                - windowSum > k
            ) {

                windowSum -= nums[left];
                left++;
            }
        }
    }
}
```

```
        }

        answer = Math.max(
            answer,
            right - left + 1
        );
    }

    return answer;
}
```

10. Time & Space Complexity

Time Complexity

Sorting:

$O(n \log n)$

Sliding Window:

$O(n)$

Overall:

$O(n \log n)$

Space Complexity

$O(1)$

Ignoring sorting space.

11. Common Mistakes / Edge Cases

Mistake 1

Not sorting the array.

Sliding window works only after sorting.

Mistake 2

Trying to decrease values.

Allowed operation:

Only Increment

Mistake 3

Integer overflow.

Use:

long

in Java.



12. Detailed Dry Run (Step-by-Step Table)

Input:

nums=[1,2,4]

k=5

After sorting:

[1,2,4]

Right	Window	Sum	Cost	Valid
0	[1]	1	0	Yes
1	[1,2]	3	$2 \times 2 - 3 = 1$	Yes
2	[1,2,4]	7	$4 \times 3 - 7 = 5$	Yes

Answer:

3

13. Common Use Cases (Real-Life / Interview)

Salary Equalization

Increase lower salaries to match a target.

Resource Allocation

Balance resources among teams.

Data Normalization

Adjust values toward a common level.

Cloud Scaling

Allocate additional capacity to reach uniform service levels.

14. Common Traps (Important!)

- Forgetting to sort
- Wrong cost formula
- Using $O(n^2)$ simulation
- Overflow in multiplication
- Shrinking window incorrectly

15. Builds To (Related LeetCode Problems)

1. LC 3 — Longest Substring Without Repeating Characters
2. LC 1004 — Max Consecutive Ones III



3. LC 424 — Longest Repeating Character Replacement
4. LC 1493 — Longest Subarray of 1's After Deleting One Element
5. LC 1838 — Frequency of the Most Frequent Element

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	$O(n^2)$	$O(1)$	TLE
Prefix Sum + Binary Search	$O(n \log n)$	$O(n)$	Good
Sorting + Sliding Window	$O(n \log n)$	$O(1)$	Optimal

17. Why This Solution Works (Short Intuition)

After sorting, the largest element in the window becomes the target value. Using the window sum, we can efficiently calculate how many increments are needed to make every element equal to that target. If the cost exceeds k , we shrink the window.

18. Variations / Follow-Up Questions

Follow-Up 1

What if decrements are also allowed?

Median-based solutions become useful.

Follow-Up 2

What if k changes for every query?

Use Prefix Sums + Binary Search.

Follow-Up 3

What if elements arrive as a stream?

Use dynamic sliding-window structures.